Video Presentation Script (English)

Project: BLM 4140 — Wi-Fi Performance Analysis **Length target:** ~10 minutes **Speaking style:** Conversational, in front of a screen with the slides or the project files open.

Numbers in brackets like `[1:30]` show the approximate point on the timeline so you can keep an eye on the clock while recording.

[0:00] Intro

Hi everyone. I'm Ozan Orhan, student number 21011077, and this is my final project for BLM 4140, Wireless and Mobile Networks, with Dr. Mehmet Şükrü Kuran.

In this project I looked at how a Wi-Fi network's total throughput changes when more and more devices try to upload data at the same time, using the NS-3 network simulator. The big question is: does the old Bianchi result — that throughput drops when the network gets crowded — still happen when we turn on all the modern 802.11n features like A-MPDU, Block-ACK and TXOP? And does the well-known RTS/CTS handshake actually help or hurt?

In the next ten minutes I will walk you through what I did, show the simulation code, and then go through the figures and tables one by one.

[1:00] Background — what is the question?

Back in 2000, G. Bianchi published a paper about the original 802.11 DCF mechanism. He showed mathematically that when more stations contend for the same channel, the probability of two stations transmitting at the same time goes up, so collisions happen more often. Each collision wastes air time, so the *total* throughput goes down even though there are more transmitters.

The catch is that Bianchi's analysis is from 2000 and only covers the old DCF. Modern Wi-Fi has EDCA (the QoS version of DCF), TXOP, A-MPDU which packs many MPDUs into one transmission, and Block-ACK. With all these turned on, you might think the Bianchi effect goes away — and that's the question this project answers.

I also compare two MAC mechanisms in every scenario:

- Plain EDCA, no RTS/CTS,
- EDCA with RTS/CTS turned on, where every transmission starts with a short reserve-the-channel handshake.

The textbook intuition is that RTS/CTS hurts when there are few stations (it adds overhead with no benefit) and helps when there are many stations (it makes collisions shorter and cheaper). I wanted to check whether that intuition still holds for an uplink-heavy TCP scenario with modern MAC features.

[2:30] Important PHY correction

One thing before we dive in. The project description PDF said "IEEE 802.11ac 2.4 GHz PHY". But 802.11ac is defined only in the 5 GHz band, so a 2.4 GHz 802.11ac PHY does not exist. The instructor confirmed this in his Google Classroom post on May 14 and asked us to use the 802.11n 2.4 GHz PHY with 11n rates instead. So everywhere in my project I use 802.11n. With 20 MHz, one spatial stream and the short guard interval, the highest MCS is MCS 7 at 72.2 Mbit/s — that's the reference rate for all my load percentages.

[3:00] Simulation setup — what I built in NS-3

I wrote a single C++ file, `wifi-project.cc`, that sits in NS-3's `scratch/` folder. It takes three command-line parameters:

- `numSTA` — 4, 8, 12 or 20 stations
- `macMechanism` — either `EDCA` or `RTSCTS`
- `totalLoadPercent` — 50, 80 or 90 percent of the 72.2 Mbit/s raw rate

The topology is simple: one AP at the origin and the STAs placed in a 1-metre circle around it, so they're all very close and they can all hear each other. There's no hidden-node problem.

For traffic I use uplink TCP. Every STA runs an OnOff application with the TCP socket factory, and the per-STA rate is the total offered load divided by the number of STAs. So the *aggregate* demand is the same regardless of how many stations I have, which is what the project description asks for.

Each scenario runs for ten seconds of simulated traffic. I run every scenario five times with five different random seeds — so that's 24 scenarios times 5 seeds equals 120 simulator invocations. All the results in my report are the mean across the five seeds, with error bars showing one standard deviation.

[4:00] How the four MAC features are turned on

Let me quickly show the four key lines in the code:

- **EDCA** is automatic. The moment I say `wifi.SetStandard(WIFI_STANDARD_80211n)`, the AP and STAs use the QoS MAC that implements EDCA.
- **A-MPDU** is enabled by setting `BE_MaxAmpduSize` to 65535 bytes on both the STA MAC and the AP MAC.
- **Block-ACK** is mandatory when A-MPDU is on, so the MAC turns it on by itself.
- **TXOP** is the one I had to set explicitly. The default TXOP-limit for AC_BE is zero, which means each channel access carries only one PPDU. I set it to 3.2 milliseconds via `BE_Txop/TxopLimits` so the AP and STAs can burst several aggregated PPDUs per transmission opportunity.
- **RTS/CTS** is controlled by `RtsCtsThreshold`. For the RTS/CTS scenarios I set it to 100 bytes (essentially every frame triggers RTS/CTS); for the plain EDCA scenarios I set it to 65535 (off).

[5:00] Running everything

To run all 120 simulations I use a tiny bash script called `run_all.sh`. It loops over seeds, STA counts, MAC mechanisms and load levels, and each simulator run writes one CSV line to `results_raw.csv`. On my machine the whole sweep takes about 15-20 minutes thanks to the optimized NS-3 build profile.

Then `plot_results.py` reads that CSV, averages across seeds, and produces four PNG figures plus three Markdown tables that I paste into the report.

[5:30] Results — Figure 1, 50 % offered load

Now let's look at the figures one by one.

Figure 1 is the lightest load, 50 percent of the raw rate, so the aggregate demand is about 36 Mbit/s. At low to moderate STA counts both curves stay between 30 and 34 Mbit/s — close to but not exactly at the 36 Mbit/s line, because some throughput is lost to MAC overhead and the AP's downstream TCP ACKs.

The interesting feature is that the EDCA and RTS/CTS curves are almost on top of each other for 4, 8 and 12 STAs. The error bars are big at 4 STAs, around 4 or 5 Mbit/s. In practice this means at low load you can't really tell the two mechanisms apart — the choice doesn't matter much.

At 20 STAs both curves drop sharply, and RTS/CTS ends up a little above EDCA. So even at moderate load, simply having 20 stations contending is enough to start eroding throughput.

[6:30] Results — Figure 2, 80 % offered load

Figure 2 is the more interesting one. We're now at 80 percent offered load, about 58 Mbit/s demand, which is already past what the network can saturate. So what we measure is the maximum the MAC can deliver, not what the applications wanted.

And here is the surprise. RTS/CTS beats plain EDCA at *every* single STA count. At 4 stations the gap is +8.5 Mbit/s. At 8 stations it's +7.9. At 12 stations it's +4.7. At 20 stations it's +5.7. That goes against the textbook rule that says RTS/CTS hurts at low N.

Why does this happen? Because with uplink TCP, every transmission is a long A-MPDU. Collisions between long A-MPDUs are extremely expensive — they waste milliseconds of air. RTS/CTS replaces those with cheap short RTS collisions. And there's a second reason: the AP itself needs to send TCP ACKs back to the STAs, and RTS/CTS helps the AP win contention against 4 to 20 uploaders.

Both curves drop with N — EDCA loses 40 percent of its throughput from 4 to 20 stations, and RTS/CTS loses 38 percent — but RTS/CTS stays uniformly higher.

[7:30] Results — Figure 3, 90 % offered load

Figure 3 is the strongest case. Now offered load is 65 Mbit/s, the network is fully saturated, and the Bianchi-style drop is at its maximum. Plain EDCA loses 39.5 percent of its throughput going from 4 to 20 stations — almost exactly the kind of monotonic curve DCF predicts.

RTS/CTS dominates everywhere again, but here the gap *grows* with N. It starts at +6.3 Mbit/s at 4 stations and grows to +9.65 Mbit/s at 20 stations, which is about a 34 percent improvement. RTS/CTS also degrades more slowly: only 28 percent from 4 to 20 stations, against 40 percent for plain EDCA. The error bars are tight, around 1 to 4 Mbit/s, and the two curves clearly don't overlap.

[8:00] Overview figure and statistical confidence

Figure 4 puts all six curves on one canvas — blue for EDCA, red for RTS/CTS, line style for the load. Three things jump out: no curve ever goes up, the red curves sit above the blue ones at 80 and 90 percent load, and at 50 percent load they are basically the same.

About robustness: each data point is the mean of 5 independent simulator runs, and the error bars are the sample standard deviation. For most operating points the stdev is under 3 Mbit/s, and the RTS/CTS advantage at 80 and 90 percent load is several times bigger than the noise. I also ran an earlier calibration with 3 seeds — the mean shifted by less than 1 Mbit/s for most points when I added the extra 2 seeds, and the EDCA vs RTS/CTS ranking never flipped. So 5 seeds are enough.

[9:00] Conclusion — what did I learn?

So what does this all add up to?

First, Bianchi's 25-year-old result is still true. Even with all of 802.11n's modern MAC features turned on, throughput goes down as the number of contending stations grows. With plain EDCA we lose 31 percent at 50 percent load, and about 40 percent at 80 and 90 percent load.

Second, RTS/CTS is surprisingly useful under heavy uplink TCP. The classic teaching says RTS/CTS hurts at low station counts, but that's based on idealized saturated UDP. With uplink TCP and modern A-MPDU, RTS/CTS beats plain EDCA at every STA count once the load hits 80 percent, with the advantage reaching plus 9.65 Mbit/s or plus 34 percent at 20 stations.

Third, at low load the choice of MAC doesn't matter much.

The practical takeaway is that in a dense, uplink-heavy 802.11n network — say, a classroom of 20 students all uploading at the same time — turning RTS/CTS on is a clear win once the network gets busy.

[9:45] Thank you

That’s all I have. Thank you for watching. The full report, the simulation source code, the raw CSV results and all four figures are in the project ZIP. If you have any questions, please email me. Thanks.